Assignment 2

100%

3.9.2024

Poengsum frakoblet modus:
100%

Legg til kommentar

Anonym vurdering: **nei**

13.10.2024

Detaljer

Introduction

You are to design and implement a calculator using HTML, CSS (with Bootstrap) and JavaScript. The calculator will only run in the browser, and must not depend on any server side scripting (C#).

Create and deliver the project on your personal git repository under the directory "assignment_2".

Tip: Don't just follow a "JavaScript calculator" tutorial. You will learn more if you figure out how to solve it yourself.

Create project

With .NET you can also use command line tools to setup and build projects. Here's how you create a project for this assignment using the command line:

```
# Navigate to the git directory you configured earlier
cd ikt201g24h/assignments/solutions

# Create a new ASP.NET Core MVC project without authentication
dotnet new mvc --auth None -o assignment_2
```

You can now open this project in Rider to code, build and run.

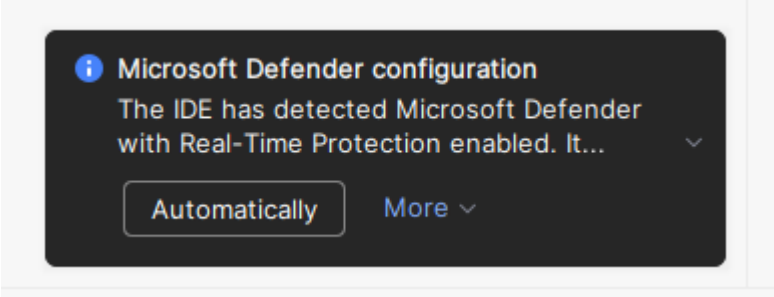
(You can also create the project from Rider, as shown in assignment 1.)

Rider setup

When you open a project for the first time there are two things you should do in Rider:

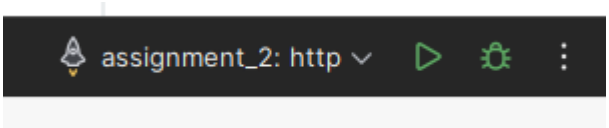
1. Accept the antivirus exclusion

This adds the project to the antivirus exclusion list, which prevents it from being scanned. This both speeds up compiling, and prevents the antivirus from blocking unknown files.



2. Set the run mode to http

There's a dropdown with multiple ways of running the project at the top of the Rider UI. Set this to http or https. The default does not work without additional software.



Functional requirements

- A display that shows entry and results
- Number buttons on the calculator
- Supports resetting the calculator (C function)
- Supports clearing the current entry (CE function)
- Handles user errors (e.g. pressing "5++5" leads to "5+5", pressing "5+-5 leads to "5-5")
- Supports using the result in the next calculation

Since this course uses automated testing, where IDs are used to find buttons on the page, correctly labeling HTML elements with IDs is important for the tests to pass. Wrong labeling of IDs is a common error source!

- Each element of the calculator needs to be labeled with the following **HTML id attribute (https://www.w3schools.com/html/html_id.asp)** .
  - Display: *display*
  - Each number button must have the following IDs: 'zero', 'one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight', 'nine'
  - + button: 'plus'
  - - button: 'minus'
  - * button: 'multiply'
  - / button: 'divide'
  - = button: 'equals'
  - C button: 'reset'
  - CE button: 'clear'

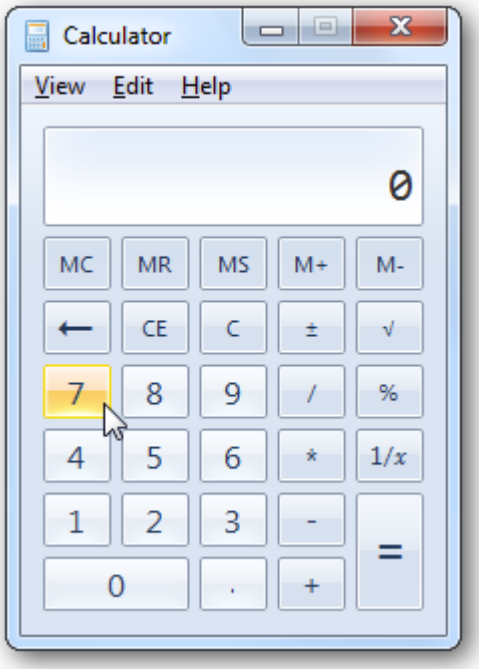
Technical requirements

- HTML, CSS and JavaScript must all be separated.
  HTML in Views/*Calculator/Index.cshtml*, CSS in css/*calculator.css* and JavaScript in js/

calculator.js in wwwroot.
Thus you must create a new controller called Calculator with a view called Index.
See the CSS/JavaScript lectures on how to add these files to the layout.

User interface

Use HTML and CSS (with Bootstrap) to design the user interface. Here's the classic Windows calculator for inspiration:



Technical implementation

There are many ways to implement a calculator, but most have a few things in common. You can build a string as the user presses a button and calculate the result on =, or you can calculate the current result immediately when a new operator (e.g. +) is pressed. Either way you need a way to evaluate what is happening.

Automatic: The easiest way to do this is to use the eval() function on the expression. I recommend solving it this way first in case you get stuck with the more advanced approach.

Advanced: A common standard way of parsing such an expression is using **infix** (https://en.wikipedia.org/wiki/Infix) to **postfix** (https://en.wikipedia.org/wiki/Reverse_Polish_notation) conversion and then use a value stack and an operator stack to calculate the result. This is more advanced and optional.

Try to organize and comment the JavaScript in a clean and tidy way.

Areas of interest

Go through some of the links in the lecture to get started. You'll also need to use **string** (http://www.w3schools.com/jsref/jsref_obj_string.asp) manipulation and/or JavaScript **arrays** (http://

www.w3schools.com/js/js_arrays.asp) , so try to research and learn about these before trying to solve the harder problem of the infix to postfix algorithm if you choose to do that.

Testing and delivery

See *Local testing* and *Delivery and Bamboo testing* under *Assignments*.